

PuppyRaffle Smart Contract — Security Audit Report

Auditor	Shweta Paranjape
Date	March 24, 2026
Target Contract	PuppyRaffle.sol
Language	Solidity ^0.8.18
Audit Type	Manual Code Review

Table of Contents

- [1. Executive Summary](#)
 - [2. Risk Classification](#)
 - [3. Findings Summary](#)
 - [4. High Severity Findings](#)
 - [5. Medium Severity Findings](#)
 - [6. Low Severity Findings](#)
 - [7. Informational / Gas Findings](#)
 - [8. Conclusion](#)
-

Executive Summary

Contract: [PuppyRaffle.sol](#) — an on-chain NFT raffle where players pay an entrance fee for a chance to win the prize pool and receive a randomly assigned rarity NFT.

Total Findings: 17

Severity	Count
High	4
Medium	3
Low	3
Informational	7

Risk Classification

Level	Description
High	Direct loss of funds, contract takeover, or permanent denial of core functionality
Medium	Indirect loss of funds, griefing, or degraded protocol fairness

Level	Description
Low	Minor logic errors, edge-case bugs, or off-chain misinterpretation risk
Informational	Best-practice violations, code quality, or gas optimisation suggestions

Findings Summary

ID	Title	Severity
H-1	Reentrancy in <code>refund</code> allows draining all contract ETH	High
H-2	Unsafe <code>uint256 -> uint64</code> cast causes <code>totalFees</code> overflow, locking fees	High
H-3	Weak on-chain randomness in <code>selectWinner</code> and rarity assignment	High
H-4	Winner's reverting fallback permanently blocks prize distribution	High
M-1	$O(n^2)$ duplicate check in <code>enterRaffle</code> enables Denial of Service	Medium
M-2	Strict <code>==</code> in <code>withdrawFees</code> causes permanent fee lockout	Medium
M-3	MEV / frontrunning in <code>refund</code> and <code>selectWinner</code>	Medium
L-1	<code>getActivePlayerIndex</code> returns <code>0</code> ambiguously for missing players	Low
L-2	Missing events and unindexed fields for critical state changes	Low
L-3	Floating pragma <code>^0.8.18</code> should be locked	Low
I-1	<code>raffleDuration</code> should be <code>immutable</code>	Informational
I-2	Image URI strings should be <code>constant</code>	Informational
I-3	Magic numbers in <code>selectWinner</code> should be named constants	Informational
I-4	Cache <code>players.length</code> outside loops to save gas	Informational
I-5	Zero-address check missing for <code>feeAddress</code> in constructor	Informational
I-6	<code>_isActivePlayer</code> is dead code — never called	Informational
I-7	<code>selectWinner</code> does not follow the CEI pattern	Informational

High Severity Findings

[H-1] Reentrancy in `PuppyRaffle::refund` allows a player to drain all contract ETH

Severity: High

Description

The `refund` function sends ETH to the caller before zeroing out the player's slot in the `players` array, violating the Checks-Effects-Interactions (CEI) pattern. A malicious contract can re-enter `refund` via its `receive` function before `players[playerIndex]` is ever cleared.

```
function refund(uint256 playerIndex) public {
    address playerAddress = players[playerIndex];
    require(playerAddress == msg.sender, "PuppyRaffle: Only the player can refund");
    require(playerAddress != address(0), "PuppyRaffle: Player already refunded, or is not active");

    // @audit ETH sent BEFORE state is updated
    payable(msg.sender).sendValue(entranceFee);

    players[playerIndex] = address(0); // too late
    emit RaffleRefunded(playerAddress);
}
```

Impact

A malicious contract can drain the entire contract balance, stealing entrance fees from all other active players.

Proof of Concept

1. Attacker enters the raffle via a malicious contract.
2. Attacker calls `refund`.
3. During the ETH transfer, `receive()` calls `refund` again — `players[playerIndex]` is still set, so the check passes.
4. Steps 3-4 repeat until the contract balance is zero.

```
contract ReentrancyAttacker {
    PuppyRaffle raffle;
    uint256 attackerIndex;

    constructor(PuppyRaffle _raffle) { raffle = _raffle; }

    function attack() external payable {
        address[] memory players = new address[](1);
        players[0] = address(this);
        raffle.enterRaffle{value: msg.value}(players);
        attackerIndex = raffle.getActivePlayerIndex(address(this));
        raffle.refund(attackerIndex);
    }

    receive() external payable {
        if (address(raffle).balance >= msg.value) {
            raffle.refund(attackerIndex);
        }
    }
}
```

Recommended Mitigation

Zero out the player's slot before sending ETH:

```
function refund(uint256 playerIndex) public {
    address playerAddress = players[playerIndex];
    require(playerAddress == msg.sender, "PuppyRaffle: Only the player can refund");
    require(playerAddress != address(0), "PuppyRaffle: Player already refunded, or is not active");

    players[playerIndex] = address(0);
    emit RaffleRefunded(playerAddress);

    payable(msg.sender).sendValue(entranceFee);
}
```

Alternatively, apply OpenZeppelin's `ReentrancyGuard` to `refund`.

[H-2] Unsafe cast of `uint256` fee to `uint64` causes `totalFees` overflow, permanently locking accumulated fees

Severity: High

Description

In `selectWinner`, the calculated fee is narrowed to `uint64` before accumulation:

```
totalFees = totalFees + uint64(fee);
```

`uint64` max value is `18,446,744,073,709,551,615` (~18.44 ETH in wei). Once cumulative fees exceed this, the addition wraps silently and `totalFees` becomes far smaller than the actual contract balance.

Impact

`withdrawFees` enforces strict equality:

```
require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are currently players active!");
```

After overflow, this require reverts permanently. All protocol fees are unrecoverable.

Proof of Concept

```
uint64 max = 18,446,744,073,709,551,615
fee        = 20 ETH in wei
uint64(fee) = 1,553,255,926,290,448,384 (wraps around)
```

```
-> totalFees << address(this).balance  
-> withdrawFees() reverts forever
```

Recommended Mitigation

Change `totalFees` to `uint256` and remove the cast:

```
uint256 public totalFees = 0;  
// ...  
totalFees = totalFees + fee;
```

[H-3] Weak on-chain randomness in `selectWinner` and rarity assignment is exploitable

Severity: High

Description

Winner selection and NFT rarity both hash predictable on-chain values:

```
uint256 winnerIndex =  
    uint256(keccak256(abi.encodePacked(msg.sender, block.timestamp,  
    block.difficulty))) % players.length;  
  
uint256 rarity =  
    uint256(keccak256(abi.encodePacked(msg.sender, block.difficulty))) %  
    100;
```

`block.timestamp` and `block.difficulty` are known or manipulable by validators. `msg.sender` is fully controlled by the caller.

Impact

- Validators can manipulate `block.difficulty` to influence the winning outcome.
- Anyone can simulate the hash off-chain and only submit `selectWinner` when the result is favourable.
- Attackers can retry transactions until they receive a Legendary NFT.

Recommended Mitigation

Use **Chainlink VRF v2** for verifiable randomness. Alternatively, implement a two-transaction commit-reveal scheme where the entropy seed is committed before it can be observed.

[H-4] Winner's reverting fallback permanently blocks prize distribution and locks all player funds

Severity: High

Description

`selectWinner` pushes ETH directly to the winner in the same transaction as the raffle reset:

```
delete players;
raffleStartTime = block.timestamp;
previousWinner = winner;

(bool success,) = winner.call{value: prizePool}("");
require(success, "PuppyRaffle: Failed to send prize pool to winner");
_safeMint(winner, tokenId);
```

If the winner is a contract without a `receive/fallback` function, the ETH push fails and the entire call reverts. Because winner selection is deterministic, every subsequent call selects the same winner and reverts again.

Impact

The raffle is permanently frozen. No new raffle can start and all player funds are locked indefinitely.

Proof of Concept

1. Smart contract wallets without `receive/fallback` enter the raffle.
2. One is selected as winner.
3. `selectWinner` reverts on every subsequent call. The raffle cannot be reset.

Recommended Mitigation

Use a pull-payment pattern — store the prize for the winner to claim separately:

```
mapping(address => uint256) public pendingWithdrawals;

// Inside selectWinner:
pendingWithdrawals[winner] += prizePool;

// New function:
function claimPrize() external {
    uint256 amount = pendingWithdrawals[msg.sender];
    require(amount > 0, "PuppyRaffle: Nothing to claim");
    pendingWithdrawals[msg.sender] = 0;
    payable(msg.sender).sendValue(amount);
}
```

Medium Severity Findings

[M-1] $O(n^2)$ duplicate check in `PuppyRaffle::enterRaffle` enables Denial of Service

Severity: Medium

Description

`enterRaffle` uses a nested loop to check for duplicates across the entire `players` array:

```
for (uint256 i = 0; i < players.length - 1; i++) {
    for (uint256 j = i + 1; j < players.length; j++) {
        require(players[i] != players[j], "PuppyRaffle: Duplicate
player");
    }
}
```

Gas cost grows quadratically as the array grows.

Impact

An attacker can fill the `players` array to make entry cost-prohibitive for legitimate participants, effectively guaranteeing their own win.

Proof of Concept

Batch	Gas Used
First 100 players	6,564,651
Second 100 players	18,937,729

The second batch costs ~3x more for the same number of entrants.

Recommended Mitigation

Replace the nested loop with an O(1) mapping lookup:

```
mapping(address => bool) public hasEntered;

function enterRaffle(address[] memory newPlayers) public payable {
    require(msg.value == entranceFee * newPlayers.length, "PuppyRaffle:
Must send enough to enter raffle");
    for (uint256 i = 0; i < newPlayers.length; i++) {
        require(!hasEntered[newPlayers[i]], "PuppyRaffle: Duplicate
player");
        hasEntered[newPlayers[i]] = true;
        players.push(newPlayers[i]);
    }
    emit RaffleEnter(newPlayers);
}
```

[M-2] **withdrawFees** uses strict `==` instead of `>=`, enabling permanent fee lockout via force-sent ETH

Severity: Medium

Description

```
require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are currently players active!");
```

ETH can be force-sent to any contract via **selfdestruct**, bypassing the absence of a **receive** function. Even 1 wei sent this way causes `address(this).balance` to exceed `totalFees`, permanently breaking fee withdrawal.

Impact

An attacker can permanently lock all protocol fees by sending 1 wei via **selfdestruct**.

Recommended Mitigation

```
require(address(this).balance >= uint256(totalFees), "PuppyRaffle: There are currently players active!");
```

[M-3] MEV and frontrunning in **refund** and **selectWinner**

Severity: Medium

Description

Two vectors exist:

- **refund**: A player who observes a **selectWinner** call in the mempool can frontrun it with **refund** after calculating they will not win, exiting before the raffle concludes.
- **selectWinner**: The winning index is deterministically computable from observable on-chain data. Searchers can simulate the result off-chain and only submit when the outcome is favourable, or validators can reorder transactions to influence the winner.

Impact

Sophisticated actors can extract value from ordinary players, undermining the fairness of the protocol.

Recommended Mitigation

- Use Chainlink VRF to remove outcome determinism (also resolves H-3).
 - Consider a commit-reveal entry scheme to prevent strategic refund timing.
 - Route sensitive keeper calls through a private mempool such as Flashbots Protect.
-

Low Severity Findings

[L-1] `getActivePlayerIndex` returns `0` ambiguously for both the first player and a missing player

Severity: Low

Description

```
function getActivePlayerIndex(address player) external view returns
(uint256) {
    for (uint256 i = 0; i < players.length; i++) {
        if (players[i] == player) return i;
    }
    return 0;
}
```

A return of `0` could mean the player is at index 0, or that the player does not exist. A refunded player at index 0 (slot set to `address(0)`) still causes `0` to be returned, giving callers no reliable way to distinguish the two cases.

Recommended Mitigation

Return a boolean alongside the index:

```
function getActivePlayerIndex(address player) external view returns (bool
found, uint256 index) {
    for (uint256 i = 0; i < players.length; i++) {
        if (players[i] == player) return (true, i);
    }
    return (false, 0);
}
```

[L-2] Missing and unindexed events for critical state changes

Severity: Low

Description

- `changeFeeAddress` updates a critical protocol parameter with no event emission.
- `RaffleRefunded` has no `indexed` field, making it impossible to filter logs by player address off-chain.

Recommended Mitigation

```
event FeeAddressChanged(address indexed oldAddress, address indexed  
newAddress);  
event RaffleRefunded(address indexed player);
```

[L-3] Floating pragma ^0.8.18 should be locked

Severity: Low

Description

```
pragma solidity ^0.8.18;
```

A floating pragma allows compilation with any 0.8.x version above 0.8.18, risking subtle behavioural differences across compiler versions.

Recommended Mitigation

```
pragma solidity 0.8.18;
```

Informational / Gas Findings

[I-1] raffleDuration should be immutable

Set once in the constructor, never modified. Mark immutable to save a storage read on every access.

```
uint256 public immutable raffleDuration;
```

[I-2] commonImageUri, rareImageUri, legendaryImageUri should be constant

Hardcoded IPFS strings that never change. Declaring them constant removes their storage slots entirely.

```
string private constant COMMON_IMAGE_URI    =  
"ipfs://QmSsYRx3LpDAb1GZQm7zZ1AuHZjfbPkD6J7s9r41xu1mf8";  
string private constant RARE_IMAGE_URI      =  
"ipfs://QmUPjADFG EKmf ohdTaNcWhp7VGk26h5jXDA7v3VtTnTLcW";  
string private constant LEGENDARY_IMAGE_URI =  
"ipfs://QmYx6GsYAKnNzZ9A6NvEKV9nf1VaDzJrqDR23Y8YSkebLU";
```

[I-3] Magic numbers in `selectWinner` should be named constants

```
// Current
uint256 prizePool = (totalAmountCollected * 80) / 100;
uint256 fee       = (totalAmountCollected * 20) / 100;

// Recommended
uint256 public constant PRIZE_POOL_PERCENTAGE = 80;
uint256 public constant FEE_PERCENTAGE       = 20;
uint256 public constant POOL_PRECISION       = 100;
```

[I-4] Cache `players.length` outside loops to avoid repeated `SLOAD`

```
// Current – SLOAD on every iteration
for (uint256 i = 0; i < players.length - 1; i++) { ... }

// Recommended – single SLOAD
uint256 playersLength = players.length;
for (uint256 i = 0; i < playersLength - 1; i++) { ... }
```

[I-5] No zero-address check for `feeAddress` in constructor

If `_feeAddress` is passed as `address(0)`, all fees are permanently lost. Add a guard:

```
require(_feeAddress != address(0), "PuppyRaffle: Fee address cannot be zero address");
```

[I-6] `_isActivePlayer` is dead code

Defined but never called anywhere in the contract. Remove it to reduce bytecode size and audit surface.

[I-7] `selectWinner` does not fully follow the CEI pattern

`_safeMint` is invoked after the ETH transfer. Since `_safeMint` calls `onERC721Received` on the recipient, this is an external call made after another external call. Move `_safeMint` before the ETH transfer, or adopt the pull-payment pattern from H-4.

Conclusion

The contract has 4 high-severity issues that must be resolved before deployment.

Priority	Finding	Action
1	H-1	Apply CEI or <code>ReentrancyGuard</code> to <code>refund</code>
2	H-2	Change <code>totalFees</code> to <code>uint256</code> , remove cast
3	H-3, M-3	Integrate Chainlink VRF
4	H-4	Switch to pull-payment for prize distribution
5	M-1	Replace nested loop with mapping
6	M-2	Change <code>==</code> to <code>>=</code> in <code>withdrawFees</code>
7	L-1 to I-7	Address in a follow-up cleanup pass

This report reflects findings from a manual review and does not guarantee the absence of all vulnerabilities. A secondary review is recommended before mainnet deployment.

Auditor: Shweta Paranjape **Date:** March 24, 2026